

This report examines the current state and future work for a CSS parser project, with emphasis on the expression evaluation and semantic analysis phases. We survey existing CSS parsing libraries and browser engines (e.g. Blink, Gecko/Stylo, PostCSS, CSSTree, LibCSS, LightningCSS), comparing their language, architecture, evaluation approaches, semantic strategies, licenses, and maturity. We extract best practices from specs and implementations for expression evaluation (AST design, operator semantics, unit handling, error handling, eager vs lazy evaluation, constant folding, precision) and for semantic analysis (type checking, property/value validation, shorthand expansion, cascade and inheritance rules, custom properties). We cover optimization strategies (memoization, incremental re-evaluation, data structures, streaming parsing, parallelism) and propose benchmarking and testing methods (microbenchmarks for calc(), var(), color conversions; use of W3C CSS Test Suite and Web Platform Tests; fuzzing and property-based tests; CI integration; profiling). We present concrete examples and pseudocode illustrating evaluator and analyzer patterns, with dos/don'ts. Finally, we outline a prioritized development roadmap with task effort estimates and recommend tools, libraries, and testing frameworks. Visualizations include a proposed CSS evaluation pipeline (Mermaid flowchart) and a Gantt-style timeline for the roadmap. All findings are grounded in official specifications (e.g. CSS Values & Units, CSS Cascading & Inheritance, CSS Variables) and authoritative sources.

Current Repository State

Note: The GitHub repository at [HazelDaniel/css-parser](#) could not be accessed for details. We therefore assume a typical CSS parser codebase. It likely contains a **lexer**, **parser** (producing an AST of CSS rules and declarations), and preliminary support for expression evaluation and semantic checks. Any missing details (e.g. exact module structure, existing tests) are assumed unspecified.

Survey of CSS Parsers and Engines

| Library/Engine   | Language          | Architecture & AST                                                                                                                                                              | Expr. Eval Approach                                                                                                                                                                                                                                                                                                                    | Semantic Strategy                                                                                                                                                                                                                                         | Maturity/Usage                               |
|------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| PostCSS          | JavaScript (Node) | Plugin-based CSS transformer; core parses rules into a shallow AST of declarations                                                                                              | Minimal built-in evaluation; value strings (parsing of values is done by plugins, e.g. <b>postcss-calc</b> , <b>postcss-colors</b> , <b>postcss-custom-properties</b> ). No formal value AST in core (plans exist to introduce                                                                                                         | Delegate to plugins; relies on plugin ecosystem for semantics (e.g. plugin checks property validity).                                                                                                                                                     | Very high (used in many tools)               |
| CSSTree          | JavaScript (Node) | Highly detailed CSS parser with tokenization and AST; uses MDN grammar data. Supports adjustable parsing detail and tolerates errors by wrapping bad fragments in Raw AST nodes | CSSTree parses values (e.g. calc(), colors, functions) into AST nodes, but does not itself perform computations. It provides a lexer to validate or match values against property syntaxes                                                                                                                                             | Provides syntax validation via its definition-syntax system (MDN/CSSWG grammars)                                                                                                                                                                          | High (widely used in tooling)                |
| LibCSS (NetSurf) | C                 | Standalone CSS parser and selection engine (from NetSurf browser). Parses CSS2/3 into a DOM-like style data structure, with a focus on fast, low-memory operation.              | Can check if a value matches a property's syntax. Does not apply semantics like cascade or inheritance.                                                                                                                                                                                                                                | Fully supports CSS expressions (length, percentage, colors, calc(), media queries, etc.). The code generator can convert values (e.g. percent to absolute length given context) as seen in release notes (e.g. supports revert, unset, new units)         | Moderate (used by NetSurf; still maintained) |
| libcroco         | C (GNOME)         | CSS2 parsing/manipulation library (GNOME); provides SAX-like API and DOM-like model                                                                                             | Parses CSS values as per CSS2 (lengths, colors, etc.). No modern calc() or custom props support.                                                                                                                                                                                                                                       | Supports cascade, inheritance, user-agent style sheets per CSS2. Does not handle CSS3/4 features.                                                                                                                                                         | Mature (ships with GNOME/GTK-based tools)    |
|                  |                   |                                                                                                                                                                                 | Full evaluation per spec: parses and resolves values (lengths, percentages, functions like calc(), var(), colors, gradients, etc.) according to the CSS specs. Supports CSS3/4 features (media queries, selectors, variables).                                                                                                         | Full CSS cascading: collects declarations, applies specificity and inheritance, computes specified and computed values, defaulting/inheriting rules. Implements value sanitization (invalid values dropped), transitions, etc.                            |                                              |
| Gecko (Mozilla)  | C++ (and Rust)    | Browser CSS engine (nsCSSParser, nsStyleSet, etc.). Tokenizes and parses CSS into internal data structures; computes style for the DOM.                                         | Performs full cascade: selector matching, specificity, inheritance, defaulting, shadow DOM style, etc. Computed styles use style struct sharing (so siblings with same styles share memory).                                                                                                                                           | Implements cascade and selection: it collates all applicable declarations, resolves specificity and important, and computes computed style for each element. Defaulting, inheritance, and shorthand decomposition are all handled in its selection engine | Very high (shipped in Firefox)               |
| Blink (Chromium) | C++               | Chrome's rendering engine: CSS parser (CSSParser and CSSParserImpl), tokenizes and produces CSOM. Uses optimized paths (CSSParserFast Paths) for common values                  | Parses and evaluates values (lengths, percentages, colors, calc(), var(), etc.). Handles quirks (quirks-mode, vendor values) and media queries.                                                                                                                                                                                        |                                                                                                                                                                                                                                                           | Very high (shipped in Chrome/Edge)           |
| Stylo (Servo)    | Rust              | High-performance CSS engine (Quantum CSS) for Servo/Firefox                                                                                                                     | Fully supports modern CSS (values from all modules: calc(), min(), custom props, media queries, etc.). Uses the cssparser Rust crate for initial parsing. Implements the CSS cascade and inheritance in parallel: does selector matching, importance, specificity. Uses style struct sharing so many elements share common style data. |                                                                                                                                                                                                                                                           | Very high (used in Firefox Nightly onward)   |

Table 1: Comparison of notable CSS parsers/engines (language, architecture, eval/semantics approach, license, and maturity). Citations highlight example documentation or code.

Expression Evaluation Best Practices

- **AST Design:** Represent values with a typed AST. For example, nodes for numbers (with units), percentages, lengths, colors, functions (calc, var, rgb, etc.), and operators. Avoid keeping raw strings; parse values into a syntax tree or token list (as in CSSTree or LightningCSS). This makes evaluation and validation systematic.
- **Evaluation Order & Operator Semantics:** Follow CSS specs for operator precedence. In calc(), \* and / bind tighter than + and -. Parentheses can group expressions. Implement standard arithmetic semantics: addition/subtraction and multiplication/division on compatible types. The CSS spec defines how to mix units: e.g. in calc(500px + 50%), the percentage is resolved relative to context so both become absolute lengths before adding. (Converting 50% of a 1000px container yields 500px; thus calc(500px+50%) = 1000px.)
- **Units & Conversions:** Internally, represent quantities as a number plus unit. At compute time, convert between compatible units (e.g. all absolute lengths to a canonical unit like px). Handle percentage arithmetic by deferring to context (percentage of what?). In cascading engines, this means holding the percentage until layout, then converting. In evaluation tools, you may require passing a reference dimension or banning % arithmetic except where spec allows (CSS defines when combining % is valid). For performance, you might postpone percent resolution until layout, treating % as a distinct type.
- **Error Handling:** Be robust to bad input. Two approaches: strict (throw error on invalid syntax) or resilient (wrap bad tokens in a special Raw AST node and continue). Resilient parsers (like CSSTree) preserve as much of the stylesheet as possible, which is useful in editors/linters. In either case, report syntax errors clearly. During evaluation, handle out-of-range operations (e.g. divide by zero) and unsupported unit combos gracefully (e.g. drop the value or treat it as invalid).
- **Eager vs Eager Evaluation:**
  - **Lazy:** Immediately compute constant subexpressions at parse time (constant folding). For example, calc(10px + 5px) can be folded to 15px. Eager folding reduces work later. However, be careful: expressions involving percentages, var(), or functions like round() might need to wait.
  - **Lazy:** Evaluate expressions only when enough context is available. This is crucial for CSS var(): custom properties are resolved after the cascade is complete (they can inherit or be overridden). Also media queries and calc() involving % often need layout info. In practice, parse into an evaluable form and postpone final computation until semantic analysis or layout.
- **Flaw Check:** Over-eager folding can violate CSS semantics (e.g. folding before cascade can lock in a wrong inherited value). The spec describes careful timing: e.g. percent arithmetic in calc() uses final used values.
- **Constant Folding & Caching:** Precompute subexpressions that involve only literals (e.g. numeric arithmetic, static colors). Cache results of repeated computations. For instance, if a calc() subtree recurs exactly (identical units and numbers), reuse the result. Memoize custom-property resolutions per element to avoid re-traversing declarations. (In a browser, style struct sharing achieves a similar goal.)
- **Numeric Precision:** Use a numeric type (e.g. double) with care. CSS Values spec ensures that precision errors are bounded: e.g. calc(pow(30px/1px,3),1/3) \* 1px is guaranteed to yield a length very close to 30px. In practice, use double precision or rational arithmetic to minimize drift. When comparing floats (e.g. during testing), allow a small epsilon.
- **CSS Value Types:** Support all CSS value kinds: Integers, reals, lengths, percentages, angles, times, frequencies, resolutions, colors (hex, rgb/rgba, hsl/hsla, named), URLs, strings, and function-call types (calc(), min(), max(), clamp(), var(), etc.). For colors, convert to an internal RGBA vector or similar for evaluation and comparison. For example, parse rgb(1/hsl) to a canonical color representation. Treat CSS-wide keywords (inherit, initial, unset, revert) specially (they skip normal evaluation).
- **Custom Properties (var()) and Cascade:** Custom properties are cascading variables. Evaluation of var(--foo) happens at computed-value time: look up the cascaded value of --foo on the element or ancestors, replacing var() with that value (or a fallback). Detect cycles (the spec spells out cycle handling: invalidating the cycle). Because of their dynamic nature, treat custom props lazily: do not fold var() in calc() until after variable resolution. A common pattern is: 1) parse calc() into AST including var() nodes; 2) after computing cascaded custom-prop values, recursively replace var() nodes with actual values; 3) then evaluate the numeric expression.
- **Operator Semantics Example (Pseudocode):**

```
function evalCalc(node, context) {
  // node is a parsed calc AST (operators and operands)
  if (node is Add) {
    let a = evalCalc(node.left, context);
    let b = evalCalc(node.right, context);
    // Convert units if needed (e.g. percentage -> px using context)
    then add numeric values, return new (value, unit)
    return addValues(a, b);
  }
  // similarly for Subtract, Multiply, Divide with spec rules...
  if (node is Var) {
    return context.resolveVariable(node.name, node.fallback);
  }
  if (node is Number or Length or Percentage) {
    return node; // literal
  }
}
```

Do: Use recursive AST evaluation with context for units and variables. Don't: Stringly-parse or reorder before tree evaluation, which can break precedence rules or percent semantics.

Semantic Analysis Best Practices

- **Property-Value Validation:** After parsing, check each declaration's value against the property's accepted syntax (as defined by the CSS spec). Ideally use a formal grammar or token-matching (e.g. the syntax definitions from CSS specs or <color> to test if the parsed value is valid for that property. Tools like CSSTree provide a matchProperty(propertyName, valueAST) utility. Invalid values should be rejected or flagged (often ignored by browsers: they drop declarations they don't understand). For example, background-color: foo; is invalid and should be skipped.
- **Type Checking:** CSS has value spaces (like <color>, <length-percentage>, enumerations). Each property has a type of values it accepts. The CSS Values & Units spec defines e.g. <length-percentage> combines lengths and percentages, or <color>, etc.. In semantic analysis, ensure numeric units are only used where allowed (e.g. don't allow px on the z-index property which takes integers; don't allow percentages in color). Use a table of property syntaxes (from specs) to enforce this.
- **Shorthand Expansion:** Many properties are shorthands (e.g. margin, border, font). During semantic phase, expand shorthand values into their constituent longhands. For example, margin: 1px 2px; should set margin-top=1px, margin-right=2px, margin-bottom=1px, margin-left=2px. This is often done in style-computation (as in browser engines) or as a transformation step in a parser. It ensures each longhand property has a canonical value for cascade/inheritance.
- **Cascade and Specificity:** Implement the cascade sorting rules: collect all declared values for each property on an element, then sort by origin (user-agent < author < user, or vice versa if important rules are present) and specificity. The highest priority wins. In code, this typically means assigning each declaration a tuple (important-flag, specificity, order) and choosing max.
- **Inheritance and Defaulting:** For properties that inherit by default (e.g. color, font-family), fill in missing values from parent elements. For non-inherited properties, use the property's initial value from the spec (e.g. display initial is inline). CSS cascade spec details inheritance and initial values. Implement the inherit and initial keywords to force these.
- **Computed Value Rules:** After cascade, compute the final computed value. This may involve unit conversion: e.g. em and % are computed relative to the relevant context (font size of parent, size of containing block). The specs "Value Processing" sections define how to normalize values: lengths to px, percentages to a number, etc. For example, font-size: calc(100% - 2px) computes 100% (parent font size in px) minus 2px. Ensure to apply these rules carefully.
- **Custom Properties Resolution:** Once cascade yields the cascaded value for each custom property (e.g. --), substitute var() references with those values in the CSS declarations. If a variable is undefined or produces a cycle, fallback handling should occur (using the var(..., fallback) value) or the declaration is invalidated as per spec. For example:

```
/* foo (-x: 10px);
.bar { width: calc(var(--x) * 2); }
```

The computed width for .bar should use .foo's --x if .bar inherits from .foo's context. Cyclic examples (the spec shows --one: calc(var(--two)+20px); --two: calc(var(--one)-20px);) must be broken by ignoring one of the definitions.

- **Interaction with Layout:** Some properties affect layout; ensure computed values are in appropriate units. For instance, width: 50% becomes a percentage of the containing block width. The evaluation engine should output a type that the layout system can use (often a float or a layout-specific value). If implementing ahead of layout, document how percentages and em will later be resolved.

Optimizations

- **Algorithmic (Memoization/Incremental):** Cache results of expensive computations (e.g. store evaluated values for each var(-prop) per element). In a dynamic setting, support incremental re-evaluation: when one declaration changes (or a DOM element is added), only re-compute affected elements. This is akin to "dirty tracking" in browser engines. Also, combine selectors intelligently (e.g. reuse matching results).
- **Data Structures:** Use efficient structures: a hashtable or map for custom properties and declarations, a specificity-sorted list for each element's declarations, AST node classes for values, etc. For selector matching, use optimized query structures (browsers use optimized algorithms).
- **Streaming Parsing to Eval:** If possible, integrate evaluation into parsing. For example, as soon as a calc() expression is parsed and if it has no variables, fold it immediately. This avoids storing full AST for simple cases. Some parsers parse and evaluate in one pass (e.g. "fast paths" in Blink). The trade-off is complexity vs. AST clarity.
- **Time Complexity:** Be aware of worst-cases: naive rule application is O(N\_elements \* N\_rules). Use optimized matching. Expression eval is generally linear in AST size. Media query evaluation and custom property resolution are typically linear. JIT for linear or near-linear overall complexity.
- **AJIT Opportunities:** Not common for CSS parsing, but if embedding in a JS environment, one might JIT-compile frequent computations (e.g. turn a parsed calc() into a faster numerical function). More applicable in CSS-to-JS engines than static parsers.

Benchmarking & Testing

- **Microbenchmarks:** Develop targeted benchmarks for critical features:
- **calc():** complex arithmetic (nested calc(), trigonometric or power functions if supported).
- **var():** large chains of custom properties and deep inheritance.
- **Color conversions:** parsing and converting many colors (hex, rgb(), hsl()).
- **Large Stylesheets:** measure end-to-end parse+eval on real-world CSS (e.g. Bootstrap or large frameworks).
- **Media/Feature queries:** scenarios with many conditional rules. Measure both throughput (bytes/sec) and latency (ms per stylesheet).
- **Performance Metrics:** Time (parse time, evaluate time), memory usage (AST size, runtime memory), and correctness (test pass/fail count). Compare against known tools (benchmark LightningCSS vs csstree vs PostCSS, etc.).
- **Test Suites:** Leverage established CSS test suites:
- **W3C CSS Test Suite:** legacy test files for CSS1/2/3 features (selectors, values, cascade). See [W3C CSS Test Suite Documentation](#).
- **Web Platform Tests (WPT):** The modern cross-browser test collection includes extensive CSS tests (e.g. cascade, selectors, variables). Use the [WPT](#) harness to run relevant CSS tests.
- **CSS Validator Tests:** The [W3C CSS Validator](#) test suite (via public APIs or the source code) can validate syntax compliance.
- **Fuzzing & Property-Based Testing:** Generate random or semi-random CSS (e.g. using QuickCheck-like tools). For each random rule, compare results with a reference implementation (e.g. a browser's output via layout dumps, or a stable library). Test invariants: e.g. calc(a+a) == 2\*a, or setting and then getting a style yields the same value. Use syntax generators based on CSS grammars to explore corner cases.
- **Regression Tests & CI:** Whenever bugs are fixed or new features added, add a test. Use continuous integration (e.g. GitHub Actions) to run style and unit tests on each commit. Integrate linters/validators to catch syntax issues early.
- **Profiling Tools:** To find bottlenecks, use profilers. In C++ code, use gprof or perf; in Rust, use perf or instrumentation crates; in Node, use Chrome DevTools or the --prof option. For example, profile parsing a large stylesheet to see which stages dominate time.

Examples, Code Patterns, and Guidelines

Expression Evaluator Pattern

```
// Pseudocode for evaluating an expression AST node
function evalNode(node, context) {
  switch(node.type) {
    case 'Number': return (value: node.value, unit: node.unit);
    case 'Addition':
      let a = evalNode(node.left, context);
      let b = evalNode(node.right, context);
      // Convert units to common base, then add
      let [aConv, bConv] = convertToCommon(a, b, context);
      return (value: aConv.value + bConv.value, unit: aConv.unit);
    case 'Multiply':
      let a = evalNode(node.left, context);
      let b = evalNode(node.right, context);
      // handle only number*value or length*number, etc.
      return multiValue(a, b);
    case 'VarFunction':
      return context.resolveVar(node.name, node.fallback);
    case 'Func': // e.g. rgb(), min(), etc.
      let args = node.args.map(arg => evalNode(arg, context));
      return handleFunction(node.name, args);
    // ...
  }
}
```

Do: Use recursion and helper functions (addValues, multiplyValues) to handle units correctly. Don't: Ignore unit mismatches or reorder operands incorrectly.

Semantic Analyzer Pattern

```
function analyzeDeclaration(prop, valueAST, context) {
  // Validate type compatibility
  if (!matchesPropertySyntax(prop, valueAST)) {
    context.errors.push(`Invalid value for ${prop}`);
    return;
  }
  // Handle shorthands: expand 'border' to 'border-width/style/color'
  if (!isShorthand(prop)) {
    let longhands = expandShorthand(prop, valueAST);
    longhands.forEach(decl => applyDeclaration(decl, context));
  } else {
    applyDeclaration({property: prop, value: valueAST}, context);
  }
}
// applyDeclaration would insert into the cascade data structures for the element.
```

Do: Check matchesPropertySyntax(prop, valueAST) against official grammars. Don't: Assume any token sequence is valid without checking.

Dos and Don'ts

- Do build a clear AST for values. Don't treat values as opaque strings.
- Do follow CSS spec rules for units and functions (see CSS Values and Units). Don't implement your own non-standard math or unit system.
- Do evaluate var() only after cascade to get final variable values. Don't fold away var() references too early.
- Do propagate inheritance and handle inherit/initial/unset keywords per CSS Cascading spec. Don't overlook initial values or reset keywords.
- Do use official test suites (WPT, CSS validator tests) as a source of truth. Don't rely solely on handcrafted examples for validation.

Recommended Tools/Libraries and Testing Frameworks

| Purpose           | Tool/Library                    | Language           | Notes/Citations                                                                  |
|-------------------|---------------------------------|--------------------|----------------------------------------------------------------------------------|
| Parsing           | <b>cssparser</b>                | Rust               | Used by Stylo/LightningCSS. Efficient PEG-based CSS tokenizer.                   |
|                   | <b>postcss</b>                  | JS (Node)          | Plugin ecosystem, broad plugin support. No built-in value AST.                   |
|                   | <b>css-tree</b>                 | JS (Node)          | Detailed CSS AST, syntax.                                                        |
| Selector Matching | <b>rust-selectors</b>           | Rust               | Used by Stylo/LightningCSS, supports complex selectors (compatible with Servo).  |
|                   | <b>Sizzle</b>                   | JS                 | (Used in jQuery) fast CSS3 selector engine.                                      |
| Expression Eval   | <b>PostCSS-calc</b>             | JS                 | Plugin to evaluate/optimize calc() expressions in PostCSS.                       |
|                   | <b>node-sass (libsass)</b>      | C/C++              | Sass compiler includes a CSS calc evaluator (though Sass has its own semantics). |
| Type Checking     | <b>W3C CSS Validator</b>        | Java               | Official validator, can be used to test CSS syntax compliance.                   |
| Testing           | <b>web-platform-tests (WPT)</b> | Various (JS, HTML) | Official cross-browser test suite. Contains CSS tests (css/**).                  |
|                   | <b>CSSWG Test Suites</b>        | Various            | Official CSS WG test suite (TNG format) for many modules (deprecated format).    |
|                   | <b>Jest / Mocha + css-tree</b>  | JS                 | Unit testing framework; use css-tree.matchProperty() for validation.             |
|                   | <b>fast-check</b>               | JS                 | Property-based testing library (generate random values).                         |
|                   | <b>rust-fuzz</b>                | Rust               | Fuzzing harness for Rust (if implementing in Rust).                              |
| Benchmarking      | <b>Benchmark.js</b>             | JS                 | High-resolution JS benchmarking (if writing in JS).                              |
|                   | <b>Criterion</b>                | Rust               | Statistical benchmarking for Rust projects.                                      |

Table 2: Suggested technologies. Citations indicate usage (e.g. Lightning CSS uses cssparser, CSSTree provides matchProperty, WPT uses CSS tests).

Roadmap and Timeline

Key tasks (prioritized):

1. Expression AST & Parser Enhancements – Implement a full CSS value AST (including functions, units, colors). Estimated: Medium effort.
2. Operator Semantics & Unit System – Add evaluation code for arithmetic (+, -, \*, /), unit conversion, color parsing. Medium.
3. Custom Property Support – Integrate cascade-based var() resolution (detect cycles, apply fallbacks). High (careful spec compliance).
4. Shorthand Expansion – Write logic to expand CSS shorthands into longhands. Medium (many cases).
5. Cascade & Specificity Engine – Build data structures to gather and sort declarations by specificity and origin. High.
6. Type Checking – Incorporate CSS property syntax definitions (e.g. via csstree grammar data) to validate values. Medium.
7. Error Handling & Recovery – Detect strict vs tolerant mode (wrap in raw nodes). Low.
8. Testing Framework Integration – Set up automated tests using WPT/CSSWG. Medium.
9. Optimization Pass – Profile current implementation; optimize hotspots (memoize, simplify AST). Low (iterate after core features).

Mermaid Gantt Chart (Timeline):



Figure: Proposed development timeline (Gantt chart).

Throughout development, refer to authoritative sources: CSS Values & Units (for grammar and computations), CSS Cascading & Inheritance (for cascade, defaulting, inheritance), and CSS Custom Properties (for variable semantics). Use the [CSSWG Issue Tracker](#) and MDN for guidance on tricky behaviors.

In Summary: By following best practices drawn from existing engines and specs, and by using the recommended tools and tests, the CSS parser project can robustly implement expression evaluation and semantic analysis. The prioritized roadmap focuses first on core parsing and cascade, then on validation and testing, and finally on performance tuning.

Sources: Official documentation and code for the mentioned libraries and specs. These include the CSSWG and MDN specifications and repository READMEs.